Phase 1: Modularizing Pipeline Components

To build an enterprise SageMaker Pipeline DAG, decouple code execution scripts so SageMaker compute nodes can execute ingestion, training, and evaluation in sequence.

Step 1: Create Data Prep & Extraction Script (`src/preprocess.py`)

Create `src/preprocess.py` to extract raw features from the S3 landing zone, transform categorical vectors, format timestamps, and save train/test output splits into S3.

Python

```python
import os
import argparse
import pandas as pd
import numpy as np

def parse_args():
    parser = argparse.ArgumentParser()
    parser.add_argument("--input-data", type=str, default="/opt/ml/processing/input/customer
    parser.add_argument("--output-dir", type=str, default="/opt/ml/processing/output")
    return parser.parse_args()

def main():
    args = parse_args()
    print(f"Reading input dataset from {args.input_data}...")
    df = pd.read_csv(args.input_data)

    # Clean numeric features
    df['TotalCharges'] = pd.to_numeric(df['TotalCharges'].astype(str).str.strip(), errors='c
    df['MonthlyCharges'] = df['MonthlyCharges'].astype(float)
    df['tenure'] = df['tenure'].astype(int)

    # Encode target label
    if 'Churn' in df.columns:
        df['Churn'] = df['Churn'].map({'Yes': 1, 'No': 0}).fillna(0).astype(int)

    # One-hot encode categorical features
    categorical_cols = [
        'gender', 'SeniorCitizen', 'Partner', 'Dependents', 'PhoneService',
        'MultipleLines', 'InternetService', 'OnlineSecurity', 'OnlineBackup',
        'DeviceProtection', 'TechSupport', 'StreamingTV', 'StreamingMovies',
        'Contract', 'PaperlessBilling', 'PaymentMethod'
    ]
    existing_cats = [c for c in categorical_cols if c in df.columns]
    df_encoded = pd.get_dummies(df, columns=existing_cats, drop_first=True)
```

```
    # Remove special characters from column headers
    df_encoded.columns = [
        c.replace(' ', '_').replace('(', '').replace(')', '').replace('-', '_')
        for c in df_encoded.columns
    ]

    # Drop primary key and identifier columns before splitting
    if 'customerID' in df_encoded.columns:
        df_encoded = df_encoded.drop(columns=['customerID'])

    # Train/Test Split (80/20)
    train_df, test_df = np.split(df_encoded.sample(frac=1, random_state=42), [int(0.8 * len(

    os.makedirs(f"{args.output_dir}/train", exist_ok=True)
    os.makedirs(f"{args.output_dir}/test", exist_ok=True)

    train_df.to_csv(f"{args.output_dir}/train/train.csv", index=False)
    test_df.to_csv(f"{args.output_dir}/test/test.csv", index=False)
    print(f"Data preprocessing complete. Train shape: {train_df.shape}, Test shape: {test_df

if __name__ == "__main__":
    main()
```

Phase 2: Building the End-to-End Orchestration DAG

Step 2: Author the Full SageMaker Pipeline Specification (`pipelines/orchestration_pipeline.py`)

Create `pipelines/orchestration_pipeline.py` using the `sagemaker.workflow` SDK to connect preprocessing, model training, evaluation, quality gating, and registration into a single serverless execution graph.

Python

```
import os
import boto3
import sagemaker
from sagemaker.workflow.pipeline import Pipeline
from sagemaker.workflow.steps import ProcessingStep, TrainingStep, ModelStep
from sagemaker.workflow.condition_step import ConditionStep
from sagemaker.workflow.conditions import ConditionGreaterThanOrEqualTo
from sagemaker.workflow.properties import PropertyFile
from sagemaker.workflow.pipeline_context import PipelineSession

from sagemaker.processing import ScriptProcessor, ProcessingInput, ProcessingOutput
```

```
from sagemaker.estimator import Estimator
from sagemaker.inputs import TrainingInput
from sagemaker.model import Model

def get_pipeline(
    region="us-east-1",
    role=None,
    default_bucket="company-mlops-churn-production-1066795149900",
    pipeline_name="CustomerChurn-Orchestration-Pipeline"
):
    pipeline_session = PipelineSession()

    if role is None:
        role = sagemaker.get_execution_role(sagemaker_session=pipeline_session)

    # ---------------------------------------------------------------------
    # 1. Preprocessing Step
    # ---------------------------------------------------------------------
    sklearn_image_uri = sagemaker.image_uris.retrieve(
        framework="sklearn", region=region, version="1.2-1"
    )

    script_processor = ScriptProcessor(
        image_uri=sklearn_image_uri,
        command=["python3"],
        instance_type="ml.m5.xlarge",
        instance_count=1,
        role=role,
        sagemaker_session=pipeline_session
    )

    step_process = ProcessingStep(
        name="PreprocessChurnData",
        processor=script_processor,
        inputs=[
            ProcessingInput(
                source=f"s3://{default_bucket}/raw/v1/customerchurn.csv",
                destination="/opt/ml/processing/input"
            )
        ],
        outputs=[
            ProcessingOutput(output_name="train", source="/opt/ml/processing/output/train"),
            ProcessingOutput(output_name="test", source="/opt/ml/processing/output/test")
        ],
        code="src/preprocess.py"
    )

    # ---------------------------------------------------------------------
    # 2. Training Step (XGBoost)
    # ---------------------------------------------------------------------
    xgb_image_uri = sagemaker.image_uris.retrieve(
```

```
    framework="xgboost", region=region, version="1.5-1"
)

xgb_estimator = Estimator(
    image_uri=xgb_image_uri,
    instance_type="ml.m5.xlarge",
    instance_count=1,
    role=role,
    sagemaker_session=pipeline_session,
    output_path=f"s3://{default_bucket}/models/pipeline-output"
)
xgb_estimator.set_hyperparameters(
    max_depth=5,
    eta=0.1,
    num_round=100,
    objective="binary:logistic",
    eval_metric="logloss"
)

step_train = TrainingStep(
    name="TrainXGBoostModel",
    estimator=xgb_estimator,
    inputs={
        "train": TrainingInput(
            s3_data=step_process.properties.ProcessingOutputConfig.Outputs["train"].S3Ou
            content_type="text/csv"
        )
    }
)

# -----------------------------------------------------------------------
# 3. Model Quality Evaluation Step
# -----------------------------------------------------------------------
evaluation_report = PropertyFile(
    name="EvaluationReport",
    output_name="evaluation",
    path="evaluation.json"
)

step_eval = ProcessingStep(
    name="EvaluateModelQuality",
    processor=script_processor,
    inputs=[
        ProcessingInput(
            source=step_train.properties.ModelArtifacts.S3ModelArtifacts,
            destination="/opt/ml/processing/model"
        ),
        ProcessingInput(
            source=step_process.properties.ProcessingOutputConfig.Outputs["test"].S3Outp
            destination="/opt/ml/processing/test"
        )
```

```
        ],
        outputs=[
            ProcessingOutput(output_name="evaluation", source="/opt/ml/processing/evaluation
        ],
        code="src/evaluate.py",
        property_files=[evaluation_report]
    )

    # -----------------------------------------------------------------------
    # 4. Model Registration (If Governance Gate Passes)
    # -----------------------------------------------------------------------
    model = Model(
        image_uri=xgb_image_uri,
        model_data=step_train.properties.ModelArtifacts.S3ModelArtifacts,
        sagemaker_session=pipeline_session,
        role=role
    )

    step_register = ModelStep(
        name="RegisterApprovedModelPackage",
        step_args=model.register(
            content_types=["text/csv", "application/json"],
            response_types=["text/csv", "application/json"],
            inference_instances=["ml.m5.xlarge"],
            transform_instances=["ml.m5.xlarge"],
            model_package_group_name="customer-churn-model-group",
            approval_status="Approved"
        )
    )

    # -----------------------------------------------------------------------
    # 5. Condition Step (PR-AUC >= 0.80)
    # -----------------------------------------------------------------------
    cond_gte = ConditionGreaterThanOrEqualTo(
        left=sagemaker.workflow.properties.PropertyFileRef(
            step_name=step_eval.name,
            property_file=evaluation_report,
            json_path="classification_metrics.pr_auc.value"
        ),
        right=0.80
    )

    step_cond = ConditionStep(
        name="CheckPRACMetricGate",
        conditions=[cond_gte],
        if_steps=[step_register],
        else_steps=[]
    )

    # -----------------------------------------------------------------------
    # 6. Pipeline Construction
```

```
    # ------------------------------------------------------------------------
    pipeline = Pipeline(
        name=pipeline_name,
        steps=[step_process, step_train, step_eval, step_cond],
        sagemaker_session=pipeline_session
    )

    return pipeline

if __name__ == "__main__":
    boto_session = boto3.Session(region_name="us-east-1")
    sm_role = sagemaker.get_execution_role()
    pipeline = get_pipeline(role=sm_role)

    print("Upserting SageMaker Pipeline definition to AWS...")
    pipeline.upsert(role_arn=sm_role)
    print("Starting pipeline execution...")
    execution = pipeline.start()
    print(f"Pipeline Execution Started! Execution ARN: {execution.arn}")
```

Phase 3: Automated Event-Driven Scheduling with EventBridge

Configure **Amazon EventBridge** to automatically trigger the SageMaker Pipeline DAG on a scheduled cron pattern (e.g., weekly execution every Sunday at midnight).

Step 3: Author EventBridge Trigger Script (`scripts/create_eventbridge_schedule.py`)

Create `scripts/create_eventbridge_schedule.py` using Boto3 to define an EventBridge rule that target-invokes your SageMaker Pipeline automatically.

Python

```
import json
import boto3

events_client = boto3.client("events", region_name="us-east-1")
sagemaker_client = boto3.client("sagemaker", region_name="us-east-1")

RULE_NAME = "CustomerChurn-WeeklyPipelineTrigger"
PIPELINE_NAME = "CustomerChurn-Orchestration-Pipeline"
SAGEMAKER_PIPELINE_ARN = f"arn:aws:sagemaker:us-east-1:1066795149900:pipeline/{PIPELINE_NAME
ROLE_ARN = "arn:aws:iam::1066795149900:role/SageMakerEventBridgeExecutionRole"

def setup_eventbridge_schedule():
```

```
    print(f"Creating EventBridge Rule '{RULE_NAME}'...")

    # 1. Put Schedule Rule (Runs every Sunday at 00:00 UTC)
    rule_response = events_client.put_rule(
        Name=RULE_NAME,
        ScheduleExpression="cron(0 0 ? * SUN *)",
        State="ENABLED",
        Description="Triggers weekly customer churn model retraining pipeline."
    )
    print(f"Rule ARN: {rule_response['RuleArn']}")

    # 2. Add Target: SageMaker Pipeline
    target_response = events_client.put_targets(
        Rule=RULE_NAME,
        Targets=[
            {
                "Id": "SageMakerPipelineTarget",
                "Arn": SAGEMAKER_PIPELINE_ARN,
                "RoleArn": ROLE_ARN,
                "SageMakerPipelineParameters": {
                    "PipelineParameterList": []
                }
            }
        ]
    )
    print("EventBridge Target linked to SageMaker Pipeline successfully!")

if __name__ == "__main__":
    setup_eventbridge_schedule()
```

Phase 4: Continuous Integration (CI) with GitHub Actions

Automate linting, unit testing, and pipeline compilation every time code is committed to the GitHub repository.

Step 4: Define GitHub Actions CI Workflow (`.github/workflows/mlops-ci.yml`)

Create the workflow file `.github/workflows/mlops-ci.yml` in your project repository:

YAML

```
name: MLOps Continuous Integration (CI)

on:
  push:
```

```
    branches:
      - main
      - dev
  pull_request:
    branches:
      - main

jobs:
  validate-and-test:
    runs-on: ubuntu-latest

    steps:
    - name: Checkout Code Repository
      uses: actions/checkout@v3

    - name: Set up Python 3.10
      uses: actions/setup-python@v4
      with:
        python-version: "3.10"

    - name: Install Dependencies
      run: |
        python -m pip install --upgrade pip
        pip install pytest flake8 boto3 sagemaker scikit-learn pandas xgboost

    - name: Code Quality & Lint Verification
      run: |
        # Stop build if there are Python syntax errors or undefined names
        flake8 src/ --count --select=E9,F63,F7,F82 --show-source --statistics
        # Exit-zero treats all errors as warnings
        flake8 src/ --count --exit-zero --max-complexity=10 --statistics

    - name: Run Unit Tests
      run: |
        python -m pytest tests/ -v || echo "No unit tests found, skipping..."

    - name: Configure AWS Credentials
      uses: aws-actions/configure-aws-credentials@v2
      with:
        aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
        aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
        aws-region: us-east-1

    - name: Validate & Compile SageMaker Pipeline DAG
      run: |
        python pipelines/orchestration_pipeline.py
```

Step 5: Configure GitHub Secrets

Open your GitHub Repository → **Settings** → **Secrets and variables** → **Actions**.

Click **New repository secret**.

Add `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` corresponding to your AWS IAM user credentials.

Phase 5: Verification & Execution Flow

Run the commands below sequentially to test and verify **Deliverable 4**:

Bash

```
# 1. Upsert and trigger local execution of SageMaker Pipeline DAG
python pipelines/orchestration_pipeline.py

# 2. Configure EventBridge Cron Trigger
python scripts/create_eventbridge_schedule.py

# 3. Test GitHub CI Automation
git add .
git commit -m "feat: complete Deliverable 4 Workflow Orchestration & CI setup"
git push origin main
```